

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO1 ASSESSMENT TOOL 1 – Case Study Analysis

Course Code:	CSA10 / CSA1014	Course Title:	Software Engineering
CO Assessed:	CO1	Assessment:	Assessment Tool 1 – Case Study Analysis
Weightage:	20%	Total Marks:	25
CO1 Statement:	Apply advanced methodologies and agile project management techniques to manage software projects effectively		
Student Name:		Reg. No.:	
Date:		Duration:	60 minutes

Code of Conduct

I **J.Mohamed Aseem(192511258)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: **J.Mohamed Aseem**

Annexure A – Case Study Analysis

Instructions to Students:

Each student will be allotted **one problem statement**. Analyze the assigned problem systematically by following the steps given below. Your report should demonstrate your understanding of software engineering concepts and your ability to design an appropriate software solution.

Based on your allotted problem statement, prepare a report by answering the following questions:

1. Understand the Problem Statement

- Explain the problem in your own words.
- Identify the objectives of the proposed system.
- State the scope of the problem.

2. Identify Key Stakeholders and Challenges

- Identify all the stakeholders involved in the system.
- Explain the roles and responsibilities of each stakeholder.
- Discuss the major challenges and issues faced by the stakeholders.

3. Analyze the Current Approach

- Describe the existing system or current process.
- Identify its strengths and limitations.
- Analyze the problems or gaps that need to be addressed.

4. Propose a Suitable Solution Using Software Engineering Principles

- Design a software-based solution for the given problem.
- Describe the major functional modules of the proposed system.
- Identify the functional and non-functional requirements.
- Explain how software engineering principles such as modularity, scalability, maintainability, reliability, usability, and security are incorporated into your solution.

5. Justify the Chosen Methodology/Framework

- Select an appropriate Software Development Life Cycle (SDLC) model or software development framework (e.g., Agile, Scrum, Waterfall, Spiral, DevOps).
- Justify why the selected methodology/framework is suitable for the proposed system.
- Explain how it supports successful project development and maintenance.

6. Discuss Expected Outcomes and Limitations

- Describe the expected benefits of the proposed solution.
- Explain how the solution addresses the identified challenges.
- Discuss potential risks, implementation challenges, and limitations.
- Suggest possible future enhancements.

Question:

20. Intelligent Software Requirements Traceability Platform

Industry Context

Large software projects contain thousands of requirements, design components, source-code modules, test cases, defects, and releases. Maintaining traceability manually is difficult.

Problem Statement

Design and develop a platform that establishes traceability relationships between requirements, design artifacts, implementation components, test cases, defects, and releases.

Objectives

- Manage software requirements.
- Link requirements with design.
- Link implementation with requirements.
- Map test cases to requirements.
- Track defects.
- Generate traceability matrices.

Expected Outcomes

- Improved requirements visibility.
- Better test coverage.
- Reduced traceability gaps.
- Easier impact analysis.
- Improved software quality.

Answers:

AIM:

To design and develop an **Intelligent Software Requirements Traceability Platform** that establishes and manages traceability relationships between software requirements, design artifacts, implementation components, test cases, defects, and software releases, thereby improving requirements visibility, test coverage, impact analysis, and overall software quality.

1. UNDERSTAND THE PROBLEM STATEMENT

Problem Explanation

Large software projects contain thousands of requirements, design components, source-code modules, test cases, defects, and releases. Managing the relationships between these artifacts manually is difficult and time-consuming.

The proposed **Intelligent Software Requirements Traceability Platform** provides a centralized system to connect requirements with their corresponding design, implementation, testing, defects, and releases. It helps development teams identify traceability gaps and understand the impact of requirement changes.

Objectives

1. To manage software requirements in a centralized platform.
2. To link requirements with corresponding design artifacts.
3. To connect implementation components with requirements.
4. To map test cases to their respective requirements.
5. To track defects associated with requirements and test cases.
6. To maintain traceability across different software releases.
7. To generate requirement traceability matrices automatically.
8. To identify missing or incomplete traceability links.
9. To support impact analysis when requirements change.

10. To improve overall software quality and testing effectiveness.

Scope

The system covers requirement management, design traceability, implementation mapping, test-case mapping, defect tracking, release management, traceability matrix generation, and impact analysis.

The platform can be used by **business analysts, project managers, software developers, software testers, quality assurance teams, and release managers.**

2. IDENTIFY KEY STAKEHOLDERS AND CHALLENGES

Stakeholder	Role and Responsibility
Project Manager	Monitors requirements, project progress, risks, and releases.
Business Analyst	Collects, analyzes, and manages software requirements.
Software Architect	Creates system designs and connects them with requirements.
Developer	Implements requirements and links source-code components.
Tester / QA Engineer	Creates test cases and verifies requirements.
Defect Manager	Records, assigns, and tracks defects.
Product Owner / Client	Provides business requirements and validates system functionality.
Release Manager	Manages requirements, defects, and components associated with releases.
System Administrator	Manages users, access permissions, and system configuration.

Major Challenges

1. Large Number of Artifacts

Large projects contain thousands of requirements, design documents, source-code files, test cases, and defects, making manual management difficult.

2. Traceability Gaps

Some requirements may not be connected to design, implementation, or test cases.

3. Requirement Changes

When a requirement changes, it can be difficult to determine which design, code, and test components are affected.

4. Poor Test Coverage

Some requirements may not have corresponding test cases, resulting in incomplete testing.

5. Manual Traceability Matrix

Preparing and updating traceability matrices manually requires significant effort.

6. Defect Tracking

Defects may not be properly associated with the requirements and test cases that caused them.

7. Release Management

Maintaining traceability across multiple software versions and releases can become complex.

3. ANALYZE THE CURRENT APPROACH

Existing System

In the existing approach, requirements, design documents, source code, test cases, and defects are often maintained using separate documents, spreadsheets, issue trackers, and testing tools.

The general process is:

Requirement → Design → Implementation → Testing → Defect → Release

However, these artifacts may be stored separately, making it difficult to establish and maintain relationships between them.

Strengths

1. Simple to implement for small projects.
2. Existing tools can be used for managing individual artifacts.
3. Teams can maintain their own project records.
4. Requires limited initial infrastructure.

Limitations

1. Traceability is mostly performed manually.
2. Human errors can occur during artifact linking.
3. Missing relationships may not be identified.
4. Requirement changes are difficult to analyze.
5. Test coverage is difficult to determine.
6. Manual generation of traceability matrices is time-consuming.
7. Cross-release traceability is difficult.
8. Collaboration between different teams may be limited.

Problems / Gaps

The major gap in the current approach is the absence of a **centralized traceability platform** that connects all software development artifacts.

The proposed system addresses this gap by maintaining:

Requirement → Design → Code → Test Case → Defect → Release

in a single platform.

4. PROPOSE A SUITABLE SOLUTION USING SOFTWARE ENGINEERING PRINCIPLES

Proposed Solution

The proposed system is a centralized web-based platform that manages software requirements and establishes relationships between requirements, design artifacts, implementation components, test cases, defects, and releases.

The system provides automated traceability matrix generation and impact analysis.

Major Functional Modules

Module 1 – Requirement Management

- Create and update requirements.
- Assign unique requirement IDs.
- Set priority and status.
- Assign requirements to team members.
- Maintain requirement history.
- Approve or reject requirements.

Module 2 – Traceability Management

- Link requirements with design artifacts.
- Link requirements with source-code components.
- Link requirements with test cases.
- Link requirements with defects.
- Associate requirements with releases.
- Identify missing traceability links.

Module 3 – Test Case and Defect Management

- Create test cases.
- Map test cases to requirements.
- Record test results.
- Track defects.
- Associate defects with requirements and test cases.
- Identify untested requirements.

Module 4 – Traceability Matrix and Impact Analysis

The system automatically generates a **Requirement Traceability Matrix (RTM)**.

Requirement Design Code Test Case Defect Release

R001	D001	C001	TC001	D001	V1.0
R002	D002	C002	TC002	—	V1.0
R003	D003	C003	—	—	V1.1

The system can also identify the affected components when a requirement is modified.

Example

Requirement R001 Changed

↓

Design D001 affected

↓

Code C001 affected

↓

Test Case TC001 requires review

↓

Defect D001 requires verification

↓

Release V1.0 may be affected

This provides faster and more accurate impact analysis.

Functional Requirements

1. The system shall allow users to create and manage requirements.

2. The system shall assign unique IDs to requirements.
3. The system shall link requirements with design artifacts.
4. The system shall link requirements with implementation components.
5. The system shall map test cases to requirements.
6. The system shall track defects.
7. The system shall associate requirements with releases.
8. The system shall generate traceability matrices.
9. The system shall identify traceability gaps.
10. The system shall support impact analysis.
11. The system shall generate project reports.
12. The system shall maintain artifact history.

Non-Functional Requirements

Performance

The system should provide quick search and retrieval of project artifacts.

Security

Only authorized users should be allowed to access or modify project information.

Scalability

The system should support increasing numbers of requirements, users, projects, and artifacts.

Reliability

The system should maintain accurate and consistent traceability information.

Usability

The interface should be simple and easy to understand.

Maintainability

The system should have modular components that can be easily modified and extended.

Application of Software Engineering Principles

Modularity

The platform is divided into separate modules such as Requirement Management, Traceability Management, Test Management, Defect Management, and Reporting.

Scalability

The system can be designed to support thousands of requirements and related artifacts.

Maintainability

Modular architecture and well-organized code make the system easier to update.

Reliability

Database validation, backup mechanisms, and audit logs help maintain reliable traceability information.

Usability

Dashboards, search functions, filters, and visual relationships make the system easier to use.

Security

Role-based access control ensures that users can access only the functions and information appropriate to their roles.

5. JUSTIFY THE CHOSEN METHODOLOGY / FRAMEWORK

Selected Methodology – Agile Scrum

Agile Scrum is selected for developing the Intelligent Software Requirements Traceability Platform.

The system contains multiple independent modules and may require continuous feedback from project managers, developers, testers, and business analysts. Agile Scrum supports incremental development and continuous improvement.

Sprint-Based Development

Sprint 1

Requirement Management

Sprint 2

Traceability and Artifact Linking

Sprint 3

Test Case and Defect Management

Sprint 4

Traceability Matrix Generation

Sprint 5

Impact Analysis

Sprint 6

Dashboard and Reporting

Justification

1. **Incremental Development:** Each module can be developed and tested separately.
2. **Continuous Feedback:** Stakeholders can review every sprint.
3. **Flexibility:** Requirement changes can be incorporated into future sprints.
4. **Early Testing:** Modules can be tested immediately after development.
5. **Risk Reduction:** Problems can be identified at an early stage.
6. **Continuous Improvement:** Sprint reviews and retrospectives improve the system continuously.

6. DISCUSS EXPECTED OUTCOMES AND LIMITATIONS

Expected Outcomes

1. Improved Requirements Visibility

All requirements and their related artifacts can be accessed from a centralized platform.

2. Better Test Coverage

The system can identify requirements that do not have corresponding test cases.

3. Reduced Traceability Gaps

Missing links between requirements, design, code, testing, defects, and releases can be identified.

4. Easier Impact Analysis

The system can identify affected components when requirements are modified.

5. Improved Software Quality

Better traceability and test coverage help reduce undetected defects.

6. Reduced Manual Work

Automatic traceability matrix generation reduces manual effort.

7. Better Collaboration

Different project teams can work with a common source of traceability information.

Limitations

1. Initial migration of existing project data may be difficult.
2. Incorrect information entered by users may result in inaccurate traceability.
3. Large projects may generate a large amount of traceability data.

4. Integration with external tools may require additional development.
5. Users may require training to use the platform effectively.
6. Automated impact analysis may require human verification.
7. Security and access control must be continuously maintained.

Future Enhancements

1. AI-based automatic requirement-to-code linking.
2. NLP-based analysis of requirement documents.
3. Automatic detection of traceability gaps.
4. Predictive impact analysis.
5. Automatic test-case recommendation.
6. Git and GitHub integration.
7. Jira integration.
8. Cloud-based deployment.
9. AI-based risk prediction.
10. Advanced project quality dashboards.

CONCLUSION

The **Intelligent Software Requirements Traceability Platform** provides a centralized solution for managing relationships between requirements, design artifacts, implementation components, test cases, defects, and releases. It reduces the limitations of manual traceability by providing automated traceability matrices, test coverage analysis, defect tracking, and impact analysis.

The use of **Agile Scrum** enables incremental development, continuous stakeholder feedback, early testing, and easy adaptation to changing requirements. The proposed system can therefore improve requirements visibility, reduce traceability gaps, simplify impact analysis, and contribute to better software quality.

This structure follows the six analysis areas specified in your reference document: problem understanding, stakeholder/challenge analysis, current-system analysis, proposed software solution, methodology justification, and outcomes/limitation.

Rubrics for Calculation

Criteria	Excellent (5 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Understanding (30%)	Clearly explains the problem, objectives, and scope with complete understanding.	Explains the problem with minor omissions.	Basic understanding with limited explanation.	Poor understanding; objectives and scope are unclear or missing.
Stakeholder & Challenge Analysis (25%)	Identifies all stakeholders, their roles, and challenges with clear analysis.	Identifies most stakeholders and challenges with adequate analysis.	Identifies only a few stakeholders or challenges.	Stakeholders and system analysis are incomplete or inaccurate.
Current System Analysis (20%)	Thoroughly analyzes the existing system, strengths, weaknesses, and identifies all gaps.	Good analysis with most strengths and weaknesses identified.	Limited analysis with partial identification of issues.	Proposed solution is unclear or inappropriate; requirements are largely missing.
Proposed Software Solution & Software Engineering Principles (15%)	Proposes an innovative, feasible solution applying appropriate software engineering principles (modularity, scalability, maintainability, security, etc.).	Suitable solution with good application of software engineering principles.	Basic solution with limited application of software engineering principles.	Methodology is inappropriate, poorly justified, or not discussed.
Methodology/ Framework Justification (10%)	Selects the most suitable SDLC/model/frame work and provides strong technical justification.	Suitable methodology selected with adequate justification.	Methodology selected but justification is weak.	Outcomes, limitations, or future enhancements are missing; report lacks organization and clarity.

Criteria	Mark
Problem Understanding (30%)	/5
Stakeholder & Challenge Analysis (25%)	/5
Current System Analysis (20%)	/5
Proposed Software Solution & Software Engineering Principles (15%)	/5
Methodology/ Framework Justification (10%)	/5
TOTAL	/25